
Упражнение 7 Построяване на криви

1. Въведение

В компютърната графика кривите се използват за описване на гладки форми, контури, траектории на движение и очертания на обекти. Те са особено полезни при моделиране на повърхности, анимации, векторна графика, CAD системи и други визуализации. За разлика от правите линии, кривите позволяват по-естествено и плавно преминаване между точки, което е от съществено значение при симулиране на реални форми.

В компютърната графика кривите се изграждат като набор от точки, изчислени по математическа формула. Колкото повече точки изчислим, толкова по-гладка изглежда кривата при визуализация.

Когато кривата може да бъде изразена чрез функция от типа $y=f(x)$, тя се нарича явна (експлицитна) крива. Такива криви са сравнително лесни за рендиране, тъй като за всяка стойност на x може да се изчисли съответната стойност на y . Това позволява използването на стандартни алгоритми за чертане на линии и криви, например:

- Параболи: квадратична функция, напр. $y=ax^2+bx+c$
- Синусоиди: тригонометрични функции, напр. $y=\sin(kx+\phi)$

Този подход е удобен, но има ограничения – не всички криви могат да бъдат описани по този начин. Например, окръжност не може да бъде описана чрез функция $y=f(x)$. В тези случаи се използват параметрични представления, при които координатите се задават чрез зависимост от трети параметър (напр. t):

- $x = r \cos(t)$, $y = r \sin(t)$ — за окръжност

При моделиране на сложни форми се използват контролни точки и специални типове криви, като Безие криви, B-Spline и NURBS, които на базата на интерполация или апроксимация предлагат гъвкавост при моделиране и манипулиране на формата.

Преди да разгледаме основните алгоритми за построяване на криви, трябва да изясним някои базови понятия:

Базови (контролни) точки

Това са определени точки в пространството, които служат за определяне на формата на кривата. Кривата не е задължително да минава през всички тях, но тяхното местоположение влияе на извивката.

Интерполация

Метод, при който крива минава точно през дадените точки (например през всички контролни точки). Използва се, когато е важно кривата да свързва точно зададени позиции. Напр. интерполационен полином на Лагранж, Катмъл-Ром сплайнове.

Апроксимация

Метод, при който крива не минава точно през всичките точки, но ги използва, за да опише близка (гладка) форма. По-често се използва за изглаждане или при по-голям брой точки. Напр. криви на Безие, B-Spline и др.

Параметрична крива

Крива, при която координатите x , y (и z при 3D) зависят от параметър t :

$$x = f(t), y = g(t)$$

Позволява описване на криви, които не могат да бъдат представени като $y = f(x)$, напр. окръжности, спирали.

2. Интерполация с полином на Лагранж

Основна цел е да намерим интерполационна функция, която минава през дадени точки. Това може да се извърши с помощта на многочлена на Лагранж.

Имаме набор от точки: $\{x_0, x_1, \dots, x_n\}$ Към всяка точка x_i съответства стойност y_i . Идеята е да се състави полином, който да минава през всички тези точки. За целта се въвеждат така наречените базисни функции $l_i(x)$, които имат следните свойства:

$$l_j(x_i) = \begin{cases} 1, & \text{ако } i = j \\ 0, & \text{ако } i \neq j \end{cases}$$

Базисната функция $l_j(x)$ се изчислява чрез:

$$l_j(x) = \frac{(x - x_0)(x - x_1) \dots (x - x_{j-1})(x - x_{j+1}) \dots (x - x_n)}{(x_j - x_0)(x_j - x_1) \dots (x_j - x_{j-1})(x_j - x_{j+1}) \dots (x_j - x_n)}$$

Интерполационният полином (общата формула) има следния вид:

$$y = \sum_{j=0}^n l_j(x) \cdot y_j = \sum_{j=0}^n y_j \cdot \prod_{\substack{j=0 \\ j \neq i}}^n \frac{(x - x_i)}{(x_j - x_i)}$$

Пример:

Известни са ни следните точки:

$$\begin{aligned} (x_0, y_0) &= (-0.8, 0.3) \\ (x_1, y_1) &= (0, -0.1) \\ (x_2, y_2) &= (0.6, 0.2) \end{aligned}$$

Интерполационният полином ще има следния вид:

$$L(x) = y_0 \cdot l_0(x) + y_1 \cdot l_1(x) + y_2 \cdot l_2(x)$$

Където:

$$\ell_0(x) = \frac{(x - x_1)(x - x_2)}{(x_0 - x_1)(x_0 - x_2)} = \frac{(x - 0)(x - 0.6)}{(-0.8 - 0)(-0.8 - 0.6)} = \frac{x(x - 0.6)}{(-0.8)(-1.4)} = \frac{x(x - 0.6)}{1.12}$$

$$\ell_1(x) = \frac{(x + 0.8)(x - 0.6)}{(0 + 0.8)(0 - 0.6)} = \frac{(x + 0.8)(x - 0.6)}{0.8 \cdot (-0.6)} = \frac{(x + 0.8)(x - 0.6)}{-0.48}$$

$$\ell_2(x) = \frac{(x + 0.8)(x - 0)}{(0.6 + 0.8)(0.6 - 0)} = \frac{(x + 0.8)x}{1.4 \cdot 0.6} = \frac{(x + 0.8)x}{0.84}$$

Можем да построим полинома:

$$L(x) = y_0 \cdot \ell_0(x) + y_1 \cdot \ell_1(x) + y_2 \cdot \ell_2(x)$$

$$L(x) = 0.3 \cdot \frac{x(x - 0.6)}{1.12} + (-0.1) \cdot \frac{(x + 0.8)(x - 0.6)}{-0.48} + 0.2 \cdot \frac{(x + 0.8)x}{0.84}$$

Нека да построим кривата с помощта на OpenGL и LWJGL. За целта като основа ще използваме проекта, асоцииран с упражнение 8:

<https://github.com/donikast/GraphicsSystems/tree/Lab8>

Проектът ще бъде реализиран на базата на трите контролни точки, описани в упражнението по-горе. Ще бъде реализирана интерполация с полином на Лагранж, като ще бъде изчертана кривата, описана с намерения полином. За да се убедим, че контролните точки лежат върху намерената крива, ще добавим допълнителна геометрия и за тяхното визуализиране.

Като начало създайте клас, който да е отговорен за създаването на изграждания модел. В него добавете трите контролни точки, по които ще бъде конструирана кривата (p0, p1, p2).

```
public class LagrangeExample extends SceneObject {
```

```
    private final Vector3f p0 = new Vector3f(-0.8f, 0.3f, 0.0f);
    private final Vector3f p1 = new Vector3f(0.0f, -0.1f, 0.0f);
    private final Vector3f p2 = new Vector3f(0.6f, 0.2f, 0.0f);
```

```
    public LagrangeExample() {
        buildExample();
    }
```

```
    private void buildExample() {

    }
```

```
@Override
```

```
public void update() {  
    }  
}
```

Създайте клас с геометрията на изгражданата крива. Предвидени са 100 върха, с помощта на които да се изчертае свързана линия.

```
public class LagrangeCurveGeometry extends Geometry {  
  
    private final int POINTS = 100; //100 върха на изчертаване  
  
    public LagrangeCurveGeometry(Vector3f p0, Vector3f p1, Vector3f p2) {  
        setData(createVertices(p0, p1, p2));  
        setAttributes(new VertexAttribute[]{  
            new VertexAttribute(0, 3),  
            new VertexAttribute(1, 3)  
        });  
        setStride(6 * Float.BYTES); // 3 for position + 3 for color  
    }  
  
    private float[] createVertices(Vector3f p0, Vector3f p1, Vector3f p2) {  
        // Базови точки  
  
        float[] data = new float[POINTS*6];  
  
        for (int i = 0; i < POINTS; i++) {  
            float x = -0.9f + i * (1.8f / (POINTS - 1)); // да се изобразят в интервала [-0.9, 0.9]  
  
            float l0 = (x - p1.x) * (x - p2.x) / ((p0.x - p1.x) * (p0.x - p2.x));  
            float l1 = (x - p0.x) * (x - p2.x) / ((p1.x - p0.x) * (p1.x - p2.x));  
            float l2 = (x - p0.x) * (x - p1.x) / ((p2.x - p0.x) * (p2.x - p1.x));  
  
            float y = p0.y * l0 + p1.y * l1 + p2.y * l2;
```

```

        data[i * 6] = x;

        data[i * 6 + 1] = y;

        data[i * 6 + 2] = 0f; //z

        data[i * 6 + 3] = 0f; //r

        data[i * 6 + 4] = 1f; //g

        data[i * 6 + 5] = 0f; //b

    }

    return data;

}

}

```

Създайте клас с геометрия на контролните точки:

```

public class ControlPointsGeometry extends Geometry {
public ControlPointsGeometry(Vector3f p0, Vector3f p1, Vector3f p2) {
    setData(createVertices(p0, p1, p2));
    setAttributes(new VertexAttribute[]{
        new VertexAttribute(0, 3),
        new VertexAttribute(1, 3)
    });
    setStride(6 * Float.BYTES); // 3 for position + 3 for color
}

private float[] createVertices(Vector3f p0, Vector3f p1, Vector3f p2) {
    return new float[]{
        // x y z r g b

        p0.x,p0.y, 0f, 1f, 0f, 0f,
        p1.x,p1.y, 0f, 1f, 0f, 0f,
        p2.x,p2.y, 0f, 1f, 0f, 0f,
    };
}
}

```

В класа LagrangeExample допълнете метода buildExample().

```

private void buildExample() {

Mesh curveMesh = new Mesh(DrawMode.LINE_STRIP, new LagrangeCurveGeometry(p0, p1, p2));
ModelElement curve = new ModelElement(curveMesh);

```

```

getModel().addElement(curve);

glPointSize(10f);
Mesh pointsMesh = new Mesh(DrawMode.POINTS, new ControlPointsGeometry(p0, p1, p2));
ModelElement points = new ModelElement(pointsMesh);
getModel().addElement(points);
}

```

Създайте LagrangeExample обект в класа SceneBuilder и стартирайте проекта за визуализация.

3. Криви на Безие

Кривите на Безие са параметрични криви, широко използвани в компютърната графика, дизайна, анимацията, CAD системи и др. Те позволяват създаване на гладки, управляеми извивки, които се контролират от няколко контролни точки.

Кривите на Безие се определят от:

- Набор от $n+1$ контролни точки: $P_0, P_1, \dots, P_n$
- Параметър $t \in [0, 1]$ – позицията по дължината на кривата
- Формула, базирана на интерполации или бинови коефициенти

Общата формула на кривата на Безие има следния вид:

$$B(t) = \sum_{i=0}^n \binom{n}{i} (1-t)^{n-i} t^i P_i$$

Където $\binom{n}{i}$ е биномен коефициент;

P_i са контролните точки

t е параметър, който променя позицията по дължината на кривата

От своя страна биномният коефициент се определя по формулата:

$$\binom{n}{i} = \frac{n!}{i!(n-i)!}$$

Където

$n!$ е факториел на n – произведението на всички цели числа от 1 до n

i е индексът на контролната точка

$(n-i)!$ е факториел на остатъка

Какъв е принципът на действие:

1. Кривата започва от P_0 и завършва в P_n
2. Междинните точки $P_1, \dots, P_{n-1}$ "издърпват" формата на кривата, но тя не минава през тях
3. Крайните точки са върху кривата, останалите – контролират извивката

Според степенния си ред кривите биват линейни, квадратични, кубични, и от по-висок порядък.

Степен	Брой контролни точки	Формула	Характеристика
1 - линейна	2	$(1-t)P_0 + tP_1$	Правя линия
2 - квадратична	3	$(1-t)^2P_0 + 2(1-t)tP_1 + t^2P_2$	Параболична линия
3 - кубична	4	$(1-t)^3P_0 + 3(1-t)^2tP_1 + 3(1-t)t^2P_2 + t^3P_3$	Гладка S-образна крива
n - от по-висок порядък	≥ 5	$\sum \binom{n}{i} (1-t)^{n-i} t^i P_i$	По-сложна форма

Пример с квадратична крива:

Нека да изберем контролни точки със следните координати:

$$P_0=(-0.9, -0.6), P_1=(0.0, 0.9), P_2=(0.8, -0.3)$$

Формулата за квадратична крива на Безие е:

$$B(t) = (1-t)^2P_0 + 2(1-t)tP_1 + t^2P_2 \quad \text{където } t \in [0, 1]$$

Ще пресметнем стойности за: $t=0.0$ (начална точка), $t=0.25$, $t=0.5$, $t=0.75$, $t=1.0$ (крайна точка).

$$B(0.0)x = (1-0)^2 * (-0.9) + 2*(1-0)*0*0 + 0^2*0.8 = -0.9$$

$$B(0.0)y = (1-0)^2 * (-0.6) + 2*(1-0)*0*0.9 + 0^2*(-0.3) = -0.6$$

$$B(0.25)x = (1-0.25)^2 * (-0.9) + 2*(1-0.25)*0.25*0 + 0.25^2*0.8 = -0.45625$$

$$B(0.25)y = (1-0.25)^2 * (-0.6) + 2*(1-0.25)*0.25*0.9 + 0.25^2*(-0.3) = -0.01875$$

$$B(0.5)x = (1-0.5)^2 * (-0.9) + 2*(1-0.5)*0.5*0 + 0.5^2*0.8 = -0.025$$

$$B(0.5)y = (1-0.5)^2 * (-0.6) + 2*(1-0.5)*0.5*0.9 + 0.5^2*(-0.3) = 0.225$$

$$B(0.75)x = (1-0.75)^2 * (-0.9) + 2*(1-0.75)*0.75*0 + 0.75^2*0.8 = 0.39375$$

$$B(0.75)y = (1-0.75)^2 * (-0.6) + 2*(1-0.75)*0.75*0.9 + 0.75^2*(-0.3) = 0.13125$$

$$B(1.0)x = (1-1)^2 * (-0.9) + 2*(1-1)*1*0 + 1^2*0.8 = 0.8$$

$$B(1.0)y = (1-1)^2 * (-0.6) + 2*(1-1)*1*0.9 + 1^2*(-0.3) = -0.3$$

Като краен резултат получаваме следните координати:

t	B(t) (x, y) координати
0.0	(-0.9, -0.6)
0.25	(-0.45625, -0.01875)
0.5	(-0.025, 0.225)
0.75	(0.39375, 0.13125)
1.0	(0.8, -0.3)

Алгоритъм на Де Кастелжо

Алгоритъмът на Де Кастелжо изчислява точка от крива на Безие за дадено $t \in [0, 1]$ чрез рекурсивна линейна интерполация между контролни точки.

Основна логика:

1. Определяме контролните точки - P_0, P_1, P_2
2. Интерполираме между съседни точки:

$$Q_0 = (1 - t)P_0 + tP_1$$

$$Q_1 = (1 - t)P_1 + tP_2$$

3. Интерполираме между новите точки:

$$B(t) = (1 - t)Q_0 + tQ_1$$

Точката $B(t)$ лежи на кривата при стойността t .

Пример

Нека да представим алгоритъма с помощта на OpenGL и LWJGL.

Подобно на кривата чрез алгоритъма на Лагранж, тук ще използваме общ клас `ControlPointsGeometry`, за визуализация на контролните точки и ще създадем геометрия, която изчислява кривата използвайки алгоритъма на Де Кастелжо.

Аналогично на инструкциите по-горе, създайте клас `BezierExample` с контролни точки.

```
public class BezierExample extends SceneObject {

    private final Vector3f p0 = new Vector3f(0.9f, 0.6f, 0.0f);
    private final Vector3f p1 = new Vector3f(0.0f, 0.9f, 0.0f);
    private final Vector3f p2 = new Vector3f(-0.8f, 0.3f, 0.0f);

    public BezierExample () {
        buildExample();
    }
}
```



```

private void buildExample() {

}

@Override
public void update() {
}
}

```

Преуизползвайте класа с геометрията на контролните точки:

```

public class ControlPointsGeometry extends Geometry {
public ControlPointsGeometry(Vector3f p0, Vector3f p1, Vector3f p2) {
setData(createVertices(p0, p1, p2));
setAttributes(new VertexAttribute[]{
    new VertexAttribute(0, 3),
    new VertexAttribute(1, 3)
});
setStride(6 * Float.BYTES); // 3 for position + 3 for color
}

private float[] createVertices(Vector3f p0, Vector3f p1, Vector3f p2) {
return new float[]{
    // x y z r g b

    p0.x,p0.y, p0.z, 1f, 0f, 0f,
    p1.x,p1.y, p1.z, 1f, 0f, 0f,
    p2.x,p2.y, p1.z, 1f, 0f, 0f,
};
}
}

```

Създайте клас BezierCurveGeometry, като предвидете 100 върха за изчертаване на геометрията.

```

public class BezierCurveGeometry extends Geometry {

private final int POINTS = 100; //100 върха на изчертаване

public BezierCurveGeometry(Vector3f p0, Vector3f p1, Vector3f p2) {
setData(createVertices(p0, p1, p2));
setAttributes(new VertexAttribute[]{
    new VertexAttribute(0, 3),
    new VertexAttribute(1, 3)
});
}
}

```

```

    setStride(6 * Float.BYTES); // 3 for position + 3 for color
}

private float[] createVertices(Vector3f p0, Vector3f p1, Vector3f p2) {

}
}

```

В рамките на класа BezierCurveGeometry добавете метод, реализиращ алгоритъма на Де Кастелжо.

```

private Vector3f deCasteljau(float t, Vector3f[] points) {
    if (points.length == 1) return points[0];

    Vector3f[] next = new Vector3f[points.length - 1];
    for (int i = 0; i < next.length; i++) {
        float x = (1 - t) * points[i].x + t * points[i + 1].x;
        float y = (1 - t) * points[i].y + t * points[i + 1].y;
        float z = (1 - t) * points[i].z + t * points[i + 1].z;
        next[i] = new Vector3f(x, y, z);
    }

    return deCasteljau(t, next);
}

```

Добавете метода за генериране на върховете:

```

private float[] createVertices(Vector3f p0, Vector3f p1, Vector3f p2) {
    // Базаваме точка

    float[] data = new float[POINTS*6];

    // Генерираме точки по кривата на Безие
    for (int i = 0; i < POINTS; i++) {
        float t = i / (float)(POINTS - 1);
        Vector3f point = deCasteljau(t, new Vector3f[] {p0, p1, p2});

        int offset = i * 6;
        data[offset] = point.x;
        data[offset + 1] = point.y;
        data[offset + 2] = point.z;
        data[offset + 3] = 1f;
        data[offset + 4] = 1f;
    }
}

```

```

        data[offset + 5] = 0f;
    }

    return data;
}

```

В класа BezierExample добавете като елементи кривата и подадените контролни точки.

```

private void buildExample() {

    Mesh curveMesh = new Mesh(DrawMode.LINE_STRIP, new BezierCurveGeometry(p0, p1, p2));
    ModelElement curve = new ModelElement(curveMesh);
    getModel().addElement(curve);

    glPointSize(10f);
    Mesh pointsMesh = new Mesh(DrawMode.POINTS, new ControlPointsGeometry(p0, p1, p2));
    ModelElement points = new ModelElement(pointsMesh);
    getModel().addElement(points);
}

```

Създайте BezierExample обект в класа SceneBuilder и стартирайте проекта за визуализация.

4. B-spline

Сплайните са основополагаща концепция в компютърната графика, моделирането, CAD системите и числения анализ, когато трябва да се създават гладки криви или повърхнини, които минават през или около дадени точки. Сплайн е крива, съставена от последователни полиноми, свързани така, че в точките на свързване кривата е непрекъсната и има гладки преходи (често и производните съвпадат).

B-spline специален вид крива, използвана в компютърната графика, CAD системи и инженерно моделиране, която се отличава със своята гъвкавост, гладкост и локален контрол. За разлика от класическите Безие криви, които използват един полином за описване на цялата крива, B-spline кривата се състои от поредица от полиноми, свързани така, че да се образува непрекъсната и гладка крива.

Вместо да се описва с един глобален полином, B-spline кривата използва локални полиноми за всеки сегмент от кривата. Тези полиноми са от една и съща степен (напр. кубични), но всеки описва само малка част от цялата крива. Тези части се „залепят“ една за друга така, че между тях има непрекъснатост в позицията и производните (напр. гладко преминаване в скорост и ускорение).

Този подход осигурява:

- по-голяма стабилност при изчисления
- по-малък риск от „осцилации“ (нежелани вълни)

- лесна адаптация на формата без промяна на цялата крива

Едно от най-големите предимства на B-spline кривите е, че всяка контролна точка влияе само върху част от кривата, а не върху цялата ѝ дължина. Това свойство се нарича локален контрол. Например при кубична B-spline крива (степен 3), всяка точка влияе само върху 4 съседни сегмента. Ако преместим една контролна точка, ще се промени само определена част от кривата, докато останалите части ще останат непроменени. Това е много важно при интерактивно моделиране, анимации с локално движение и коригиране на криви без нарушаване на формата извън даден участък. При класическите Безие криви, броят на контролните точки определя и степента на полинома, което води до проблеми при използване на много точки (бавни изчисления и нестабилност).

С B-spline обаче:

- степента остава фиксирана (напр. 3 - кубична), независимо дали имаш 4 или 40 точки
- нови точки могат да се добавят, без да се увеличава сложността на полиномите

Това прави B-spline изключително подходяща за сложни и големи криви, които трябва да бъдат лесни за управление и бързи за рендериране.

За да бъде определена B-spline крива, са нужни следните 4 компонента:

1) Контролни точки

Това са основните точки, които насочват формата на кривата. Обозначават се с $P_0, P_1, \dots, P_n$. Броят им е $n+1$. Не всички точки лежат върху кривата.

2) Степен на кривата p

Това е степента на полиномите, от които е съставена кривата. Най-често се използва степен 3 (кубична B-spline).

3) Възлов вектор $U=\{u_0, u_1, \dots, u_m\}$

Това е ненамаляваща последователност от стойности, които задават как се разпределя параметърът t върху кривата. Възлите определят областите на влияние на всяка контролна точка.

4) Базисни функции $N_{i,p}(t)$

Те се използват за претегляне на влиянието на всяка точка P_i върху текущата точка от кривата при стойност t .

Базов случай (степен 0):

$$N_{i,0}(t) = \begin{cases} 1, & \text{ако } u_i \leq t < u_{i+1} \\ 0, & \text{иначе} \end{cases}$$

За по-висока степен p :

$$N_{i,p}(t) = \frac{t - u_i}{u_{i+p} - u_i} N_{i,p-1}(t) + \frac{u_{i+p+1} - t}{u_{i+p+1} - u_{i+1}} N_{i+1,p-1}(t)$$

Общото описание на кривата има следния вид:

$$C(t) = \sum_{i=0}^n N_{i,p}(t) \cdot P_i \quad \text{за } t \in [u_p, u_{m-p-1}]$$

Всяка точка $C(t)$ от кривата се получава чрез сума от контролни точки, претеглени с базисните функции.

Задачи за самостоятелна работа

Задача 1 Стартирайте проекти с приложените по-горе примери и наблюдавайте кривите при изменения.

- Добавете четвърта контролна точка, като актуализирате създадените алгоритми. Наблюдавайте резултата.
- По отношение на кривата на Безие заменете алгоритъма на Де Кастелжо с класическа формула на Безие.
- Актуализирайте съществуващите класове и методи по такъв начин, че те да са универсални и независими от броя на подадените контролни точки.

Задача 2 Използвайки интерполация с полинома на Лагранж, да се построи крива, описана чрез четири контролни точки: $A = (-0.7, 0.8, 0.0)$, $B = (-0.4, 0.2, 0.0)$, $C = (0.0, -0.2, 0.0)$, $D = (0.7, 0.4, 0.0)$.

Извършете следното:

1. Изчислете базисните функции $\ell_i(x)$
2. Постройте интерполационния полином $L(x)$
3. Рендирайте кривата чрез OpenGL (LWJGL) с помощта на 100 точки.

Задача 3 Да се построи крива на Безие, описана чрез три контролни точки: $P_0 = (-0.5, 0.5)$, $P_1 = (0.0, -0.7)$, $P_2 = (0.8, 0.3)$.

1. Изчислете координатите на точките от кривата при $t=0.3$, $t=0.6$ и $t=0.9$.
2. Рендирайте кривата като използвате алгоритъма на Де Кастелжо.
3. Добавете още една контролна точка с координати по избор и наблюдавайте резултата.

Задача 4 Използвайте избран от Вас алгоритъм и рендирайте изображение – контур на сърце.